

User as Engram: Internalizing Per-User Memory as Local Parametric Edits

Anonymous submission

Abstract

Personal memory combines user-specific *content* with the *reasoning skill* needed to use it. Biological memory separates local hippocampal engrams from shared neocortical skill; language-model personalization usually keeps facts outside the weights or folds both roles into a global per-user LoRA. We propose **User as Engram**: write content into the hash-keyed memory rows of an Engram model and learn reasoning once in a shared adapter. On the same controlled base, an Engram write causes about $33,000\times$ less held-out loss increase than a matched per-user LoRA and, at 100 facts, uses 88 KB rather than 14.2 MB. The edit is inspectable: it activates at its trigger, supplies the answer value, leaves other positions and weights bit-identical, and fails in the wrong layer. Disjoint user maps compose additively in one table. The layered design matches LoRA’s direct recall, improves mean indirect-reasoning accuracy by $5.6\times$, and makes no test user worse than the untouched base across our three-seed evaluation. Because addressed lookup does not search the external population, it overtakes retrieval beyond roughly 100 facts in our controlled scaling test despite using a $2.5\times$ smaller answering model.

Introduction

Personalization requires both remembering a user’s facts and using them to reason. Current systems usually keep facts in a prompt or retrieval store (Lewis et al. 2020; Gao et al. 2023), paying context and search costs on every request. Per-user adapters instead internalize facts, but a LoRA (Hu et al. 2022) is a global change to a shared function: it can perturb unrelated behavior, must be loaded separately for each user, and does not compose cleanly across tenants.

We propose *User as Engram*: store user content as local edits to the hash-addressed memory table of an Engram-pretrained Transformer (Cheng et al. 2026), while learning reasoning skill once in a shared LoRA. A fact activates only the rows selected by its suffix N -grams; all other rows and backbone weights remain bit-identical. Per-user state is therefore a small override map that can be swapped per request or combined additively with other disjoint maps. This separates rapidly changing, private content from slowly learned,

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

population-level skill, mirroring complementary biological memory systems (Semon 1921; Tonegawa et al. 2015; McClelland, McNaughton, and O’Reilly 1995).

Figure 1 illustrates this separation. The two roles have different lifecycles. A user’s doctor or preference may change and require individual deletion; inferring that a cardiologist is relevant to a heart check-up is stable and transferable. A per-user adapter conflates them, repeatedly relearning the relation and making records difficult to locate or delete. Our design keeps content explicit and reasoning reusable.

Our evidence supports four claims: a per-user LoRA causes roughly $33,000\times$ more held-out loss increase than an Engram edit; the edit’s address, gate, residual contribution, and answer-token effect are traceable; at 100 facts, joint row optimization approaches LoRA recall with 88 KB rather than 14.2 MB per user; and shared reasoning plus local content preserves direct recall while improving indirect reasoning without retrieval cost that grows with the stored population.

Background and Design Requirements

Personal-memory mechanisms span three families. Prompt memories and retrieval stores keep content editable, but consume context and select evidence at every turn (Lewis et al. 2020; Gao et al. 2023). Parameter-efficient tuning moves content into a shared function (Hu et al. 2022). Model editing targets particular associations, yet usually produces weight changes whose locality is empirical (Meng et al. 2022, 2023). Engram pretraining supplies a fourth substrate: explicit key-value rows in the residual stream (Cheng et al. 2026).

We evaluate personal-memory methods along five requirements that are easy to conflate. *Recall* asks whether the exact fact can be produced. *Use* asks whether it can participate in a novel inference. *Locality* asks whether the write leaves unrelated behavior unchanged. *Lifecycle* requires individual update and deletion. *Serving* requires many users to share one backbone. A method that succeeds on recall alone can fail all four remaining requirements; this motivates reporting held-out loss, indirect questions, address-level audits, and per-user storage alongside accuracy.

The term *Engram* here denotes the model’s pretrained hashed memory table, not an arbitrary cache added after training. Its gates have learned when table values should enter the residual stream. We exploit that learned interface but do not alter its addressing rule or backbone. Our contribution is

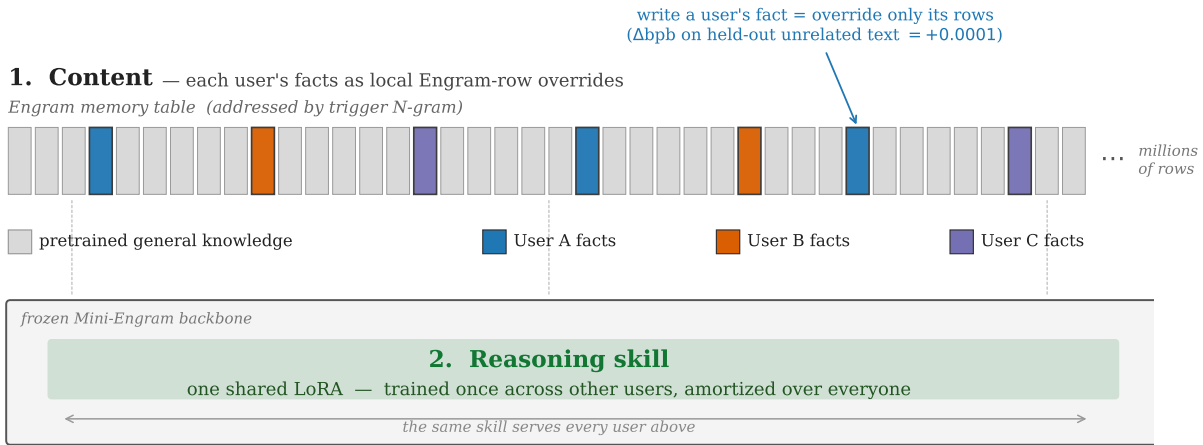


Figure 1: Local Engram content with a shared Transformer and reasoning adapter.

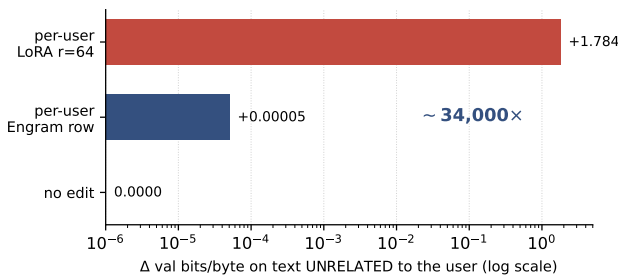


Figure 2: Held-out unrelated-text contamination for matched Engram and rank-64 LoRA edits (canonical seed, $n = 20$ users).

a write and composition protocol for per-user overrides, plus the separation of those overrides from a shared adapter that learns population-level reasoning.

Why Facts Should Not Be Per-User Weight Deltas

Weights are appropriate for shared abstractions but problematic for mutable, tenant-specific records. We isolate this distinction using Mini-Engram-d20 and the same 20 users. A rank-64 LoRA trained on one fact per user raises held-out loss by 1.784 on the canonical seed, with 17/20 users worse. Addressed Engram rows change loss by only 0.00005, with none worse. Across three seeds the means are +1.698 versus +0.000052 ($\sim 33,000\times$), and 49/60 versus 0/60 users worsen. Figure 2 shows the canonical-seed comparison.

The distinction is architectural, not merely an optimizer failure. A LoRA must move shared projections to raise the desired answer probability, so its update can fire on unrelated inputs. A keyed memory write is inert unless its address is read. Strong instruction-tuned bases can conceal the behavioral damage by already solving a question, but cannot remove the underlying loss increase.

Across Qwen2.5-3B/7B (Qwen Team 2025), Llama-3.1-8B (Grattafiori et al. 2024), and Mistral-7B (Jiang et al. 2023), only 0–20% of users score worse after personalization, and mean indirect accuracy changes by +0.088 to +0.217; on the base LM, 85% score worse and the mean is -0.133 . Instruction tuning can therefore mask the task-level effect of a global LoRA update, but the update remains nonlocal. Each per-user LoRA is still an alternative global model state, whereas sparse override maps share a backbone.

User as Engram

An Engram layer hashes token-suffix N -grams to memory rows, gates their values, and adds them to the residual stream. To write a fact, we put its answer where its trigger is read: deterministic hashes map the trigger’s suffix to a sparse address set R_f (about 16 rows for $N = 3$, $K = 8$). The backbone and every row outside R_f remain fixed. What remains is what value to place in those rows, for which we use the three strategies below. OPT backpropagates the answer loss only into R_f ; Joint OPT samples among a user’s facts while updating their row union so co-active values can coexist. A 100-fact override map occupies about 88 KB. Figure 3 shows the resulting surgical write.

The edit is a glass box. Figure 4 contrasts its footprint with the matched LoRA. For an inserted fact we can enumerate the trigger hashes, verify that only R_f changed, inspect the learned gates, measure the residual contribution at each layer, and observe the answer token rise in the logit lens. On unrelated prompts the edited rows are not addressed and the residual delta is exactly zero. By contrast, the matched LoRA alters every layer and position. Locality therefore follows from the data structure rather than from a regularizer.

We study three insertion strategies. *UNEMBED_P* solves in closed form for a row value whose projection favors the answer and costs <1 ms. *OPT* fits each arriving fact separately. *Joint OPT* revisits all facts while fitting their row union; it costs more at write time but best tolerates collisions and simultaneous activation. Their editable parameter set is

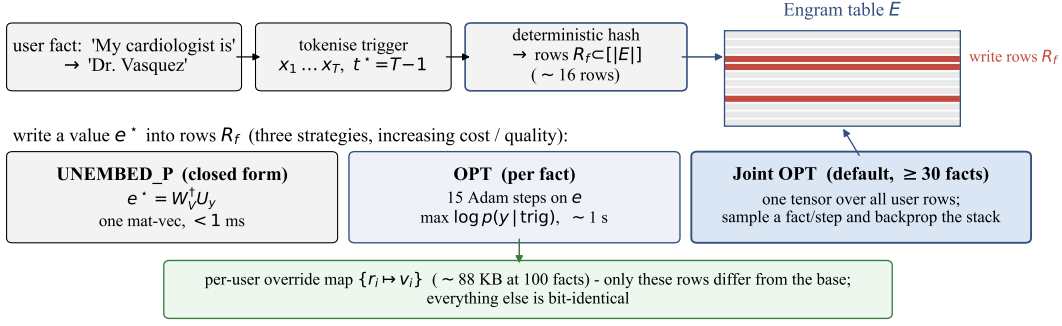


Figure 3: Surgical insertion into deterministically addressed Engram rows.

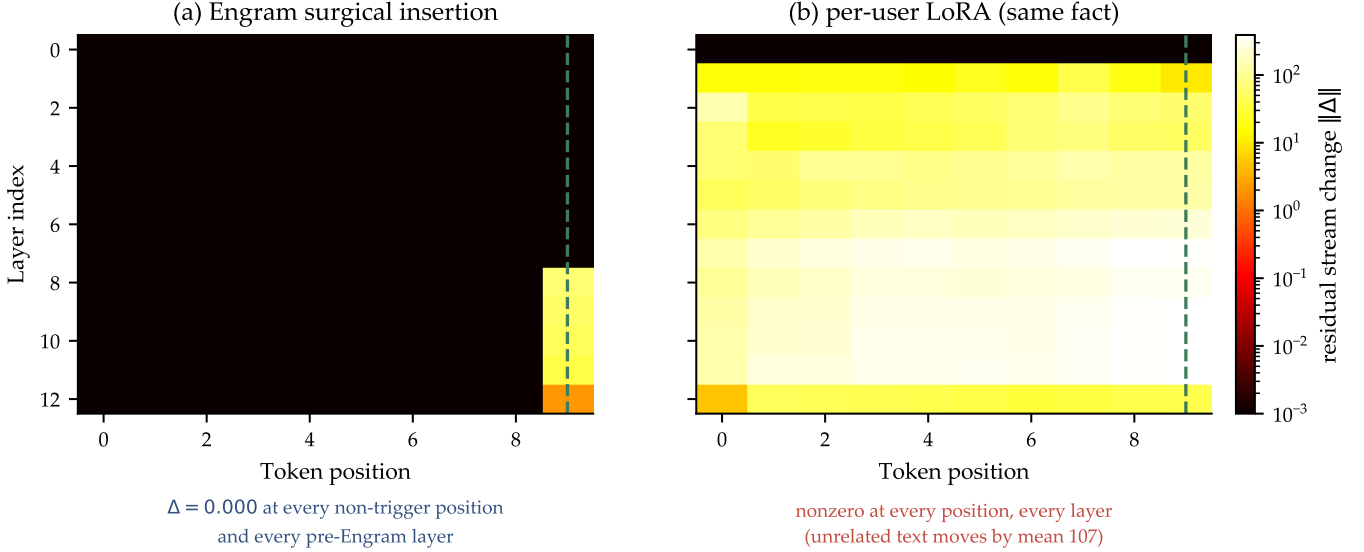


Figure 4: Residual changes from matched Engram and per-user LoRA edits.

identical, so this is a trade-off in write quality and cost, not in isolation.

Keys, correctness, and deletion. Personal questions often reproduce an entity or relation phrase even when surrounding wording changes. Multiple suffix orders and hashes provide redundant triggers; inserting paraphrased question forms expands coverage. This is not semantic search: a query with no overlapping trigger may miss. Because addresses are deterministic, we know the complete editable row set before optimization. Deleting a user removes the override map rather than reversing a gradient update; updating a fact replaces explicit entries.

Mechanistic checks. We validate locality four ways: byte comparison leaves the backbone and unaddressed rows identical; non-trigger positions have zero residual delta; the trigger gate rises from 0.02 to 0.99; and the residual change aligns with the value path at cosine 0.999. The same write placed early drops recall from near-perfect to about one quarter, confirming that late placement matters.

The comparison LoRA is optimized to the same recall target. Its residual delta appears at all positions and layers,

including an unrelated “capital of France” control, because low-rank does not mean locally activated. Norm or KL regularization might reduce that footprint, but would not make the update address-conditioned or identify exactly which inputs can activate it. For Engram, the zero-delta region follows directly from the lookup key.

Each user owns an override map $\Delta_u : h \mapsto v$. At inference, lookups first consult that map and otherwise fall back to the shared table. Maps with disjoint keys commute, so personal, team, and organizational memories can be stacked without merging dense weight deltas. Hash collisions are explicit and can be detected. In the current implementation the last write wins; maps whose triggers overlap should therefore remain separate.

Experiments

We evaluate Mini-Engrams derived from nanochat/GPT-2 (Karpathy 2026; Radford et al. 2019) against per-user LoRA, in-context facts, and retrieval, matching backbone, tokenizer, generation, and facts unless noted. Metrics cover direct and paraphrase recall, held-out LM loss, per-user storage, and indirect questions that transform stored facts.

Table 1: Mini-Engram backbones. All share the 51.2M-parameter memory table.

	d8 v2	d12 v2	d12@1280	d20@1536
depth $\times$ width	8×512	12×768	12×1280	20×1536
Engram layers	2, 5	2, 7	2, 7	2, 11
total parameters	178M	339M	625M	1.22B
ClimbMix tokens	0.50B	1.32B	3.34B	7.40B
validation bpb	.912	.827	.770	.730

Controlled bases vary depth and width but share the lookup. We report accuracy and held-out next-token loss because strong models can mask contamination once answers saturate. Storage excludes the backbone and shared adapter, which are amortized across tenants.

Synthetic schemas control recall and reasoning; LOCOMO adds conversational memory. Direct questions request stored answers, whereas indirect questions apply held-out relations and cannot be solved by echoing triggers. We repeat key contamination and layering results over three seeds. Writes are offline; query cost includes lookup, search, and context, and storage means serialized trainable state rather than duplicated checkpoints.

Protocol and comparison controls

All Mini-Engrams use one lookup rule and a $50K \times 256$ (51.2M-parameter) table; Table 1 lists their scales. We train in bf16 at 12 tokens per scaling parameter. Base, XL, and XXL corpora contain 100 USER + 100 ORG facts, 1,000 of each, and 3,132 trigger templates, respectively. At 100 facts, Joint OPT uses 2,000 Adam steps at rate 0.5. The three-seed test matches rank-64 LoRA (1,500 steps, 5×10^{-4} , $\alpha = 128$) against Joint OPT (1,500 steps, rate 0.5).

Metrics. Direct top-1 matches the first generated token to gold; `indirect_any` finds the gold span anywhere. LOCOMO uses token-F1 and an LLM judge (Zheng et al. 2023), since F1 may reward a correct first token followed by noise. Contamination is the change in held-out bits per byte (Δ bpb) on unrelated ClimbMix: 262K tokens for the canonical figure and 524,288 for three-seed runs.

The shared rank-16 adapter trains on 10 users and tests on 20 disjoint users; contamination and high-density comparisons use rank-64 per-user Q/K/V LoRA. Engram sweeps vary insertion, density, model scale, triggers, and facts; per-rank, per-seed, and capacity tables remain available for supplementary review.

No-memory measures prior knowledge. For indirect reasoning, relations are learned from other users and test content enters only through the evaluated memory, requiring unseen-content access and transferable skill. Unrelated held-out ClimbMix makes Δ bpb measure collateral change, not fact recall.

Recall, density, and storage. A single edit is within eight points of the in-context ceiling. On Mini-Engram-d12, joint optimization handles interacting addressed rows at high density. At 100 facts it approaches rank-64 LoRA with $161 \times$ less per-user storage (88 KB vs. 14.2 MB). Ownership stays sparse despite shared rows; Figure 5 supports Joint OPT at

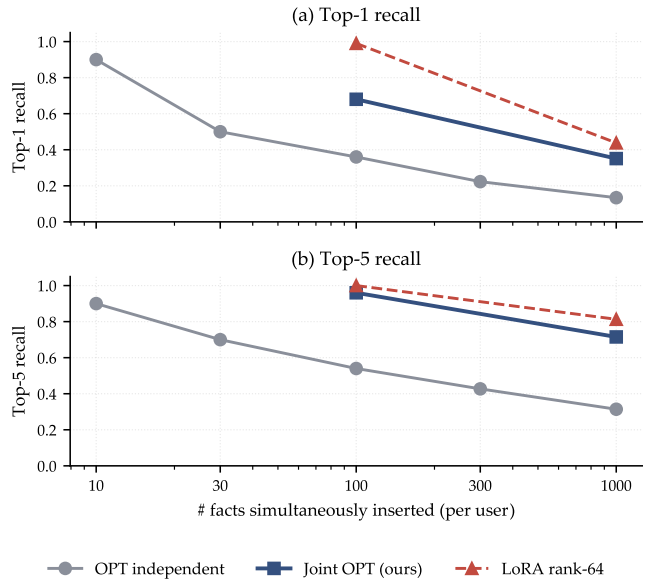


Figure 5: Top-1 and top-5 recall versus facts per override map.

≥ 30 facts and shows top-5 recall above 90% to roughly 300 facts/user.

Memory-system and LOCOMO comparisons. Nearest-neighbor retrieval is strongest when queries repeat stored triggers. Our MEM0 and MemMachine baselines use only their retrieval recipes because facts are already structured; extraction and consolidation are out of scope (Chhikara et al. 2025; Wang et al. 2026). With all-MiniLM-L6-v2 (Wang et al. 2020; Reimers and Gurevych 2019), multi-trigger Engram gains 22 paraphrase points over the best retrieval baseline without context tokens.

Single-hop LOCOMO uses the first 80 non-adversarial questions from each of 10 conversations (~ 800) (Maharana et al. 2024). Retrieval stores gold evidence turns and Engram gold Q/A pairs: a controlled best-case comparison with a fixed Mini-Engram-d12 answering LM, not an end-to-end system. Across scales, Joint OPT leads on single-hop, multi-hop, and reasoning but trails on open-domain questions, where local facts cannot replace broad knowledge (Table 2; Figure 6). LongMemEval and PersonaMem-v2 provide complementary long-term and personalized memory evaluations (Wu et al. 2025; Jiang et al. 2025).

The LLM judge puts first-token Engram behind retrieval because token-F1 can reward a correct first token followed by noise. Multi-token Joint OPT trains the full answer and restores the lead on larger models; we claim no universal LOCOMO win.

Joint OPT’s largest gains are multi-hop: token-F1 improves 120–178% over the better retrieval baseline, versus 10–58% on single-hop and reasoning. Open-domain performance falls 51–57%, favoring addressed personal memory plus retrieval for broad, changing knowledge.

The paraphrase result separates address coverage from answer quality. Retrieval is nearly perfect on repeated prefixes;

Table 2: LOCOMO token-F1 by category and model. Joint OPT is compared with the stronger retrieval baseline in each cell.

category	answering LM	MEMO	MemMachine	Engram J-OPT	lead vs. best retr.
single-hop	d12 v2	0.131	0.116	0.169	+29%
	d12@1280	0.160	0.152	0.176	+10%
	d20	0.161	0.169	0.233	+38%
multi-hop	d12 v2	0.092	0.076	0.243	+165%
	d12@1280	0.115	0.104	0.253	+120%
	d20	0.102	0.108	0.299	+178%
reasoning	d12 v2	0.117	0.113	0.183	+56%
	d12@1280	0.123	0.136	0.203	+49%
	d20	0.135	0.168	0.266	+58%
open-domain	d12 v2	0.247	0.209	0.122	−51%
	d12@1280	0.341	0.327	0.146	−57%
	d20	0.314	0.338	0.159	−53%

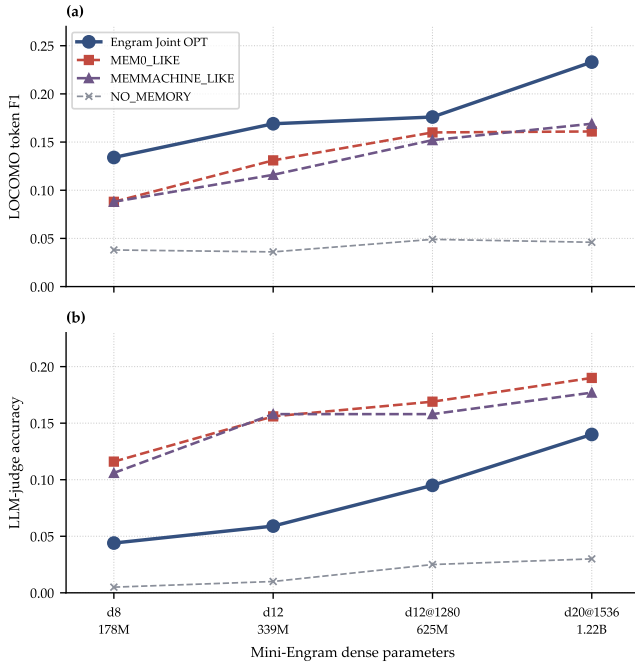


Figure 6: LOCOMO token-F1 and Qwen2.5-14B-Instruct judge accuracy across answering models.

more trigger variants improve Engram paraphrases without prompt tokens. Neither implies universal semantic generalization: a stored key must activate.

Multi-hop use. Across 63 two-fact chains, fact recall is 99.2% and direct second-fact matching is 90.6%, but true chaining falls to 12.9%: the store matches but does not compose. We therefore learn relations in a shared adapter on held-out users while fitting only test-user rows.

Interpreting the comparisons. Across Table 3’s three seeds, matched recall isolates storage location, density tests coexistence, and layering tests whether locality sacrifices use.

Recall need not cause global contamination, and reasoning can be shared; LOCOMO shows that addressed memory does not replace open-domain retrieval.

The $33,000\times$ loss ratio is a model-, fact-, and optimization-dependent stress test, not a constant. The robust distinction is qualitative: Engram can be inactive off-address, whereas LoRA changes the shared mapping. Likewise, 88 KB is the chosen 100-fact configuration, not a lower bound; width, precision, triggers, and collisions change it.

Negative results and boundary cases. UNEMBED_P is fast but underperforms when pretrained values misalign with the answer. Independent fitting degrades under trigger collisions, and joint fitting repairs only part of the high-density loss. Free continuation is harder than short-span matching; unseen paraphrase triggers can fail. An arbitrary reasoning LoRA also fails unless it learns the same content interface, motivating layered training.

Reproducibility scope. The source retains insertion-strategy, rank, capacity, per-seed, pretraining, latency, judge-prompt, and alternative-architecture details for a planned supplement; the main paper retains the results needed for the causal chain.

Shared Skill, Local Content

We train the rank-16 shared LoRA on 10 users with each indirect question’s needed facts in context, then test 20 disjoint users whose facts arrive only through Engram rows. Six conditions cross content placement (none, per-user LoRA, or Engram) with shared skill while holding direct recall nearly constant. Across three seeds, layered Engram reaches 100% versus 98.8% direct recall and 41.2% versus 7.3% indirect accuracy for per-user LoRA.

The layered system pays $+0.385 \Delta\text{bpb}$ once for the shared adapter; each user’s rows add only ≈ 0.00005 , versus $+1.698$ for a separate per-user LoRA. The adapter learns a reusable transformation while local rows supply user-specific arguments (Figure 7).

The layered-minus-LoRA indirect gain has paired-bootstrap 95% CI $[+31, +37]$ percentage points.

Table 3: Three-seed robustness on Mini-Engram-d20 ($n = 20$ users per seed).

Quantity	S0	S1	S2	Mean (range)
per-user LoRA Δ bpb	1.784	1.754	1.557	1.698 (1.56–1.78)
per-user Engram Δ bpb	.000052	.000049	.000054	.000052
layered total Δ bpb	.386	.388	.379	.385 (.38–.39)
layered indirect_any	44.5%	44.2%	34.8%	41.2% (34.8–44.5)
per-user LoRA indirect_any	6.0%	8.8%	7.2%	7.3% (6.0–8.8)
worse than base (layered / LoRA)	0/20 / 17/20	0/20 / 16/20	0/20 / 16/20	0/60 / 49/60

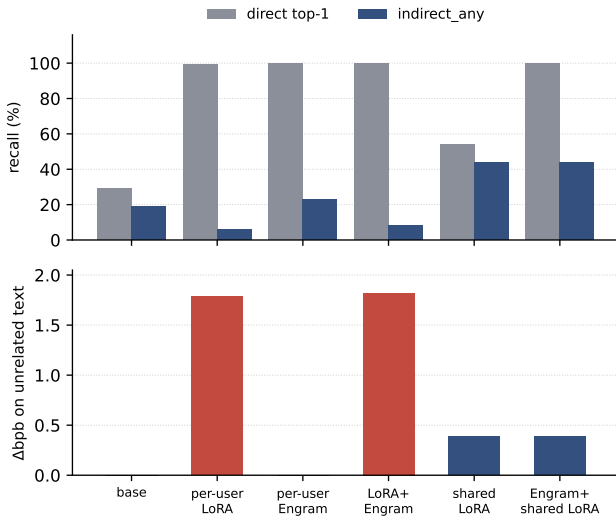


Figure 7: Direct and indirect recall and held-out Δ bpb across six conditions on Mini-Engram-d20 (seed S0, $n = 20$).

The factorial design attributes the gain: Engram without shared skill tests recall, shared skill without content tests leakage, and naive LoRA+Engram tests arbitrary stacking. Only intended layering gives local content, transferable reasoning, and low held-out loss; train and test users are disjoint.

Cross-schema transfer. The headline comparison uses disjoint users within one schema family. To test beyond those templates, we train on first-person personal facts and evaluate on third-person medical records. Layered indirect accuracy falls from 44% to 31% (30% relative), while per-user LoRA falls from 6% to 4%; the advantage persists ($7.4\times$ to $7.6\times$). Locality is unchanged: Δ bpb stays $+0.386$ because personal rows activate only at addressed triggers (Figure 8).

Retrieval narrows the gap with the larger Qwen2.5-3B-Instruct (Qwen Team 2025), but prompt and search costs grow with the knowledge base. Fixed-cost Engram overtakes it beyond roughly 100 facts despite a $2.5\times$ smaller answering model. Retrieval suits changing corpora, provenance, and open-domain evidence; Engram suits compact, frequently accessed personal state with stable triggers (Figure 9).

To isolate knowledge-base size, we add held-out schema-family distractors to each user’s 34 facts, reaching $N \in \{34, 100, 200, 300, 500, 1000\}$ with the same 20 users and

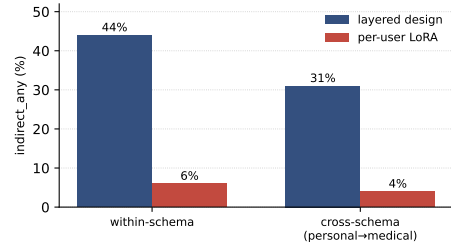


Figure 8: Within-schema and personal-to-medical transfer for layered Engram and per-user LoRA.

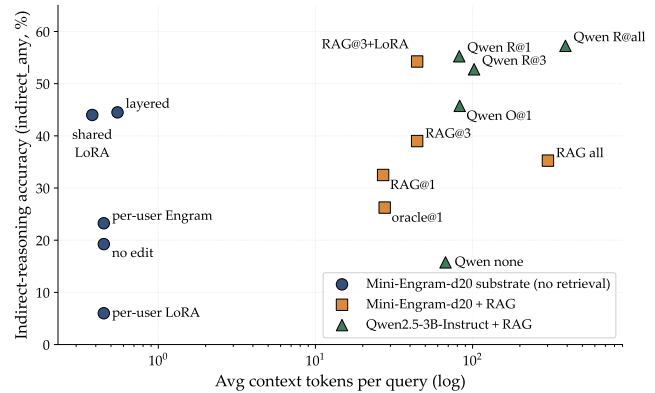


Figure 9: Indirect accuracy versus query context cost; Qwen-3B is the $2.5\times$ larger answering model.

probes. Retrieval coverage falls with distractors, whereas addressed Engram lookup is corpus-size independent (Figure 10).

Multi-Tenant Serving

A request resolves a sparse user table, saves and overrides affected base rows, runs one forward pass, then restores them. User switches change neither dense weights nor the shared adapter. Disjoint personal, team, and organizational scopes can layer; overlapping triggers are last-write-wins and should remain separate. Unmeasured batched serving would require one gather per Engram layer.

At one million users, 88 KB each is roughly 88–100 GB depending on accounting, plus one 11.8 MB shared rank-16 adapter; 14.2 MB per-user LoRAs require about 14.2 TB. Omitting allocator and index overheads, sparse content scales

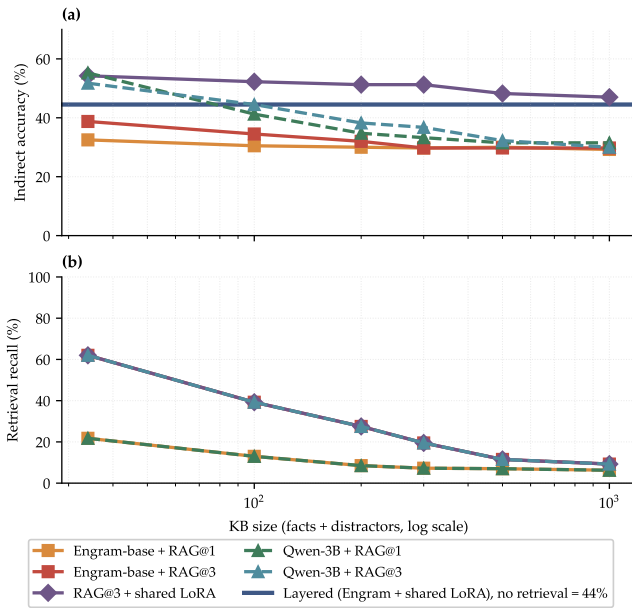


Figure 10: Indirect accuracy and retrieval recall versus knowledge-base size.

with stored facts while each LoRA scales with model dimensions.

S-LoRA and Punica (Sheng et al. 2024; Chen et al. 2024) batch dense adapters, but each tenant retains a model-shaped delta; overrides are indexed reads in the existing layer. Production requires untested caching, authorization, encryption, collision telemetry, and rate limits. The prototype demonstrates composition and scaling, not security.

The prototype uses sequential row swaps, not an optimized batched kernel. Per-request work is user-count independent, so throughput barely changes as users triple; latency still depends on cache residency and hardware (Figure 11).

Discussion, Limitations, and Related Work

At 1,000 facts Joint OPT reaches 35% top-1; with <1% slot occupancy, the ceiling is forward-pass interference among co-active rows, not collisions. Paraphrase coverage depends

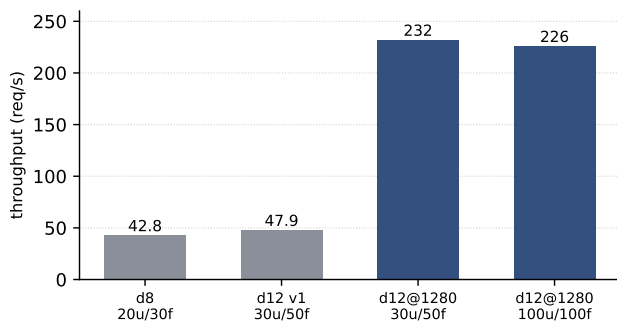


Figure 11: Prototype throughput across model, user-count, and fact-count configurations.

on triggers, and optimization is slower than appending a document. Engram pretraining is also required: lacking public weights, we trained 178M–1.22B models; the 625M run took 10.5 GPU-hours.

Write locality provides neither privacy, deletion policy, access control, nor factual verification. We also do not test whether shared reasoning encodes population biases; deployment requires auditing and systems support.

Our work connects parameter-efficient personalization and per-user adapters (Houlsby et al. 2019; Hu et al. 2022; Zhang et al. 2024; Su et al. 2025; Tan, Liu, and Jiang 2024; Zhuang et al. 2024), retrieval-augmented generation (Lewis et al. 2020; Gao et al. 2023; Borgeaud et al. 2022), model editing (Meng et al. 2022, 2023; Mitchell et al. 2022a,b; Wang et al. 2024), and external or semi-parametric memory (Lample et al. 2019; Packer et al. 2023; Cheng et al. 2026). These methods provide non-parametric access or global weight changes. Our content/skill boundary instead places mutable facts in addressed rows and reusable reasoning in shared weights, yielding locality and multi-tenancy rather than dense per-user models. The dense-adapter interference we measure is related to catastrophic forgetting in continual learning (McCloskey and Cohen 1989; Kirkpatrick et al. 2017).

Validity and societal considerations. Controlled models expose changed rows and small loss deltas, strengthening causal claims but limiting frontier-scale conclusions. Locomo mixes memory and base knowledge, so we report category reversals. Storage and latency constants are implementation-dependent, though their scaling forms are robust. Addressability aids inspection and erasure but can expose records; untested exfiltration, access control, output auditing, and shared-skill fairness remain. Generative AI tools provided editorial assistance; the authors reviewed and verified all text, claims, citations, figures, and reported results.

What should improve next. Three recipe changes target these limits: multi-fact training, random per-user hash offsets for unseen rows, and “recall, then reason” chaining examples (Sun et al. 2023). Preliminary multi-fact finetuning reached the same recall faster without raising its ceiling. Teacher traces lowered accuracy, showing that training and evaluation formats must align.

The strongest next comparison is a frontier-scale model pretrained with this interface. Small bases expose locality and failures but leave larger-scale utility untested. We expect hybrid routing: addressed memory for frequent records, retrieval for long-tail evidence and provenance, and shared weights for stable transformations.

Conclusion

Personal memory need not require a model copy. User as Engram stores facts in inspectable override maps, sharing reasoning across users to improve locality, composition, storage, and indirect reasoning while exposing when retrieval or context is preferable.

References

- Borgeaud, S.; Mensch, A.; Hoffmann, J.; et al. 2022. Improving Language Models by Retrieving from Trillions of Tokens. In *Proceedings of the International Conference on Machine Learning*.
- Chen, L.; Ye, Z.; Wu, Y.; et al. 2024. Punica: Multi-Tenant LoRA Serving. In *Proceedings of MLSys*.
- Cheng, X.; Zeng, W.; Dai, D.; Chen, Q.; Wang, B.; Xie, Z.; Huang, K.; Yu, X.; Hao, Z.; Li, Y.; Zhang, H.; Zhang, H.; Zhao, D.; and Liang, W. 2026. Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. arXiv:2601.07372.
- Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413.
- Gao, Y.; et al. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.
- Grattafiori, A.; Dubey, A.; Jauhri, A.; et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783.
- Houlsby, N.; et al. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the International Conference on Machine Learning*.
- Hu, E.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; et al. 2023. Mistral 7B. arXiv:2310.06825.
- Jiang, B.; Yuan, Y.; Shen, M.; et al. 2025. PersonaMem-v2: Towards Personalized Intelligence via Learning Implicit User Personas and Agentic Memory. arXiv:2512.06688.
- Karpathy, A. 2026. nanochat: An Experimental Training Harness for LLMs.
- Kirkpatrick, J.; Pascanu, R.; Rabinowitz, N.; et al. 2017. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences*, 114(13): 3521–3526.
- Lample, G.; Sablayrolles, A.; Ranzato, M.; Denoyer, L.; and Jégou, H. 2019. Large Memory Layers with Product Keys. In *Advances in Neural Information Processing Systems*.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.
- Maharana, A.; et al. 2024. LOCOMO: Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*.
- McClelland, J. L.; McNaughton, B. L.; and O'Reilly, R. C. 1995. Why There Are Complementary Learning Systems in the Hippocampus and Neocortex. *Psychological Review*, 102(3): 419–457.
- McCloskey, M.; and Cohen, N. J. 1989. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In *Psychology of Learning and Motivation*, volume 24, 109–165. Academic Press.
- Meng, K.; Bau, D.; Andonian, A.; and Belinkov, Y. 2022. Locating and Editing Factual Associations in GPT (ROME). In *Advances in Neural Information Processing Systems*.
- Meng, K.; et al. 2023. MEMIT: Mass-Editing Memory in a Transformer. In *International Conference on Learning Representations*.
- Mitchell, E.; Lin, C.; Bosselut, A.; Finn, C.; and Manning, C. D. 2022a. Fast Model Editing at Scale. In *International Conference on Learning Representations*.
- Mitchell, E.; Lin, C.; Bosselut, A.; Manning, C. D.; and Finn, C. 2022b. Memory-Based Model Editing at Scale. In *Proceedings of the International Conference on Machine Learning*.
- Packer, C.; Wooders, S.; Lin, K.; et al. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.
- Qwen Team. 2025. Qwen2.5 Technical Report. arXiv:2412.15115.
- Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; and Sutskever, I. 2019. Language Models Are Unsupervised Multitask Learners (GPT-2). Technical report, OpenAI.
- Reimers, N.; and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*.
- Semon, R. 1921. *The Mneme*. London: George Allen & Unwin.
- Sheng, Y.; Cao, S.; Li, D.; et al. 2024. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. arXiv:2311.03285.
- Su, W.; et al. 2025. Parametric Retrieval-Augmented Generation (PRAG). arXiv:2501.15915.
- Sun, Z.; et al. 2023. Recitation-Augmented Language Models. In *International Conference on Learning Representations*.
- Tan, Z.; Liu, Q.; and Jiang, M. 2024. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-Tuning (OPPU). In *Proceedings of EMNLP*.
- Tonegawa, S.; Liu, X.; Ramirez, S.; and Redondo, R. 2015. Memory Engram Cells Have Come of Age. *Neuron*, 87(5): 918–931.
- Wang, P.; Li, Z.; Zhang, N.; et al. 2024. WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models. In *Advances in Neural Information Processing Systems*.
- Wang, S.; Yu, E.; Love, O.; Zhang, T.; Wong, T.; Scargall, S.; and Fan, C. 2026. MemMachine: A Ground-Truth-Preserving Memory System for Personalized AI Agents. arXiv:2604.04853.
- Wang, W.; Wei, F.; Dong, L.; Bao, H.; Yang, N.; and Zhou, M. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*.

Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations*.

Zhang, Z.; Rossi, R. A.; Kveton, B.; et al. 2024. Personalization of Large Language Models: A Survey. arXiv:2411.00027.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems: Datasets and Benchmarks*.

Zhuang, Y.; et al. 2024. HYDRA: Per-User Adapters for Personalised LLMs.